

第 四 章

(第 3、4、8、10、11、12、17、20、21、24、26 题)

3. 假设某个 C 语言函数 func() 的原型声明如下:

```
void func(int *xptr, int *yptr, int *zptr);
```

函数 func() 的过程体对应的机器级代码用 AT&T 汇编形式表示如下:

```
1  movl    8(%ebp), %eax
2  movl    12(%ebp), %ebx
3  movl    16(%ebp), %ecx
4  movl    (%ebx), %edx
5  movl    (%ecx), %esi
6  movl    (%eax), %edi
7  movl    %edi, (%ebx)
8  movl    %edx, (%ecx)
9  movl    %esi, (%eax)
```

请回答下列问题或完成下列任务。

- (1) 上述机器级代码是在 IA-32 还是 x86-64 系统中生成的? 为什么?
- (2) 在过程体开始时三个入口参数对应实参所存放的存储单元地址是什么? (提示: 当前栈帧底部由帧指针寄存器 EBP 指示)
- (3) 根据上述机器级代码写出函数 func() 的 C 语言代码。

参考答案:

- (1) 机器级代码是在 IA-32 系统中生成的。因为汇编指令中出现的都是 32 位寄存器。
- (2) xptr、yptr 和 zptr 对应实参所存放的存储单元地址分别为: R[ebp]+8、R[ebp]+12、R[ebp]+16。
- (3) 函数 func 的 C 语言代码如下:

```
void func(int *xptr, int *yptr, int *zptr)
{
    int tempx=*xptr;
    int tempy=*yptr;
    int tempz=*zptr;
    *yptr = tempx;
    *zptr = tempy;
    *xptr = tempz;
}
```

4. 假设函数 operate() 的部分 C 代码如下:

```
1  int  operate(int x, int y, int z, int k) {
2      int  v = _____ ;
3      return v;
4  }
```

以下 IA-32 汇编代码用来实现第 2 行语句的功能:

```
1  movl    12(%ebp), %ecx
2  sall    $8, %ecx
3  movl    8(%ebp), %eax
4  movl    20(%ebp), %edx
```

```

5    imull    %edx, %eax
6    movl     16(%ebp), %edx
7    andl     $65520, %edx
8    addl     %ecx, %edx
9    subl     %edx, %eax

```

回答下列问题或完成下列任务。

(1) 写出每条汇编指令的注释, 并填写 operate() 函数缺失的部分。

(2) 给出函数 operate() 对应的 x86-64 汇编代码, 并和 IA-32 汇编代码进行性能比较。

参考答案:

(1) 每条汇编指令的注释如下:

```

movl 12(%ebp), %ecx    //R[ecx]←M[R[ebp]+12], 将 y 送 ECX
sall $8, %ecx          //R[ecx]←R[ecx]<<8, 将 y*256 送 ECX
movl 8(%ebp), %eax     //R[eax]←M[R[ebp]+8], 将 x 送 EAX
movl 20(%ebp), %edx    //R[edx]←M[R[ebp]+20], 将 k 送 EDX
imull %edx, %eax       //R[eax]←R[eax]*R[edx], 将 x*k 送 EAX
movl 16(%ebp), %edx    //R[edx]←M[R[ebp]+16], 将 z 送 EDX
andl $65520, %edx     //R[edx]←R[edx] and 65520, 将 z&0xFFF0 送 EDX
addl %ecx, %edx        //R[edx]←R[edx] + R[ecx], 将 z&0xFFF0+y*256 送 EDX
subl %edx, %eax        //R[eax]←R[eax]-R[edx], 将 x*k-(z&0xFFF0+y*256)送 EAX

```

根据以上分析可知, 第 2 行缺失部分为:

```

3    int  v = x*k-(z&0xFFF0+y*256) ;

```

(2) 函数 operate() 对应的 x86-64 汇编代码如下:

```

sall $8, %esi          //R[esi]←R[esi]<<8, 将 y*256 送 ESI
imull %edi, %ecx       //R[ecx]←R[ecx]*R[edi], 将 x*k 送 ECX
andl $65520, %edx     //R[edx]←R[edx] and 65520, 将 z&0xFFF0 送 EDX
addl %esi, %edx        //R[edx]←R[edx] + R[esi], 将 z&0xFFF0+y*256 送 EDX
subl %edx, %ecx        //R[ecx]←R[ecx]-R[edx], 将 x*k-(z&0xFFF0+y*256)送 ECX
movl %ecx, %eax        //R[ecx]←R[ecx], 将 x*k-(z&0xFFF0+y*256)送 EAX 作为返回值

```

因为 x86-64 采用寄存器传递参数 (x, y, z, k 分别存放在 EDI、ESI、EDX 和 ECX 中), 因此, 相对于 IA-32, 减少了 4 条从栈中读取参数到寄存器的指令 (IA-32 汇编代码中标黄的指令), 因为栈区在内存中, 所以这些指令需读取主存中数据, 因而比其他非访存指令的执行时间更长。

8. 已知函数 do_loop() 的 C 语言代码如下:

```

1    short do_loop(short x, short y, short k) {
2        do {
3            x*=(y%k);
4            k--;
5        } while ((k>0) && (y>k));
6        return x;
7    }

```

函数 do_loop 的过程体对应的 IA-32 汇编代码如下:

```

1    movw 8(%ebp), %bx
2    movw 12(%ebp), %si

```

```

3    movw    16(%ebp), %cx
4  .L1:
5    movw    %si, %dx
6    movw    %dx, %ax
7    sarw    $15, %dx
8    idiv    %cx
9    imulw    %dx, %bx
10   decw    %cx
11   testw    %cx, %cx
12   jle     .L2
13   cmpw    %cx, %si
14   jg      .L1
15  .L2:
16  movswl    %bx, %eax

```

回答下列问题或完成下列任务。

- (1) 给每条汇编指令添加注释，并说明每条指令执行后，目的寄存器中存放的是什么内容？
- (2) 上述函数过程体中用到了哪些被调用者保存寄存器和哪些调用者保存寄存器？在该函数过程体前面的准备阶段哪些寄存器必须保存到栈中？
- (3) 为什么第 7 行中的 DX 寄存器需要算术右移 15 位？
- (4) 给出 do_loop()函数对应的 x86-64 汇编代码。

参考答案：

- (1) 每个入口参数都要按 4 字节边界对齐，因此，参数 x、y 和 k 入栈时都占 4 个字节。

```

1    movw    8(%ebp), %bx      //R[bx]←M[R[ebp]+8], 将 x 送 BX
2    movw    12(%ebp), %si     //R[si]←M[R[ebp]+12], 将 y 送 SI
3    movw    16(%ebp), %cx     //R[cx]←M[R[ebp]+16], 将 k 送 CX
4  .L1:
5    movw    %si, %dx          //R[dx]←R[si], 将 y 送 DX
6    movw    %dx, %ax          //R[ax]←R[dx], 将 y 送 AX
7    sarw    $15, %dx          //R[dx]←R[dx]>>15, 将 y 的符号扩展 16 位送 DX
8    idiv    %cx               //R[dx]←R[dx-ax]÷R[cx]的余数, 将 y%k 送 DX
                                //R[ax]←R[dx-ax]÷R[cx]的商, 将 y/k 送 AX
9    imulw    %dx, %bx         //R[bx]←R[bx]*R[dx], 将 x*(y%k) 送 BX
10   decw    %cx               //R[cx]←R[cx]-1, 将 k-1 送 CX
11   testw    %cx, %cx         //R[cx] and R[cx], 得 OF=CF=0, 负数则 SF=1, 零则 ZF=1
12   jle     .L2               //若 k 小于等于 0, 则转.L2
13   cmpw    %cx, %si          //R[si] - R[cx], 将 y 与 k 相减得到各标志
14   jg      .L1               //若 y 大于 k, 则转.L1
15  .L2:
16  movswl    %bx, %eax        // R[eax]←R[bx], 将 x*(y%k) 送 AX

```

- (2) 被调用者保存寄存器有 BX、SI，调用者保存寄存器有 AX、CX 和 DX。在该函数过程体前面的准备阶段，被调用者保存的寄存器 EBX 和 ESI 必须保存到栈中。

(3) 因为执行第 8 行除法指令前必须先将被除数扩展为 32 位，这里是带符号整数除，因此，采用算术右移以扩展 16 位符号，符号扩展的 16 位放在 DX 中，低 16 位在 AX 中。

- (4) 对应的 x86-64 汇编代码如下（入口参数 x, y, k 分别在 DI、SI 和 DX 中）：

```

1    movw    %dx, %cx          //R[cx]←R[dx], 将 k 送 CX
2  .L1:
3    movw    %si, %dx          //R[dx]←R[si], 将 y 送 DX
4    movw    %dx, %ax          //R[ax]←R[dx], 将 y 送 AX
5    sarw    $15, %dx          //R[dx]←R[dx]>>15, 将 y 的符号扩展 16 位送 DX
6    idiv    %cx               //R[dx]←R[dx-ax]÷R[cx]的余数, 将 y%k 送 DX
                                //R[ax]←R[dx-ax]÷R[cx]的商, 将 y/k 送 AX
7    imulw    %dx, %di         //R[di]←R[di]*R[dx], 将 x*(y%k) 送 DI
8    decw    %cx              //R[cx]←R[cx]-1, 将 k-1 送 CX
9    testw    %cx, %cx         //R[cx] and R[cx], 得 OF=CF=0, 负数则 SF=1, 零则 ZF=1
10   jle     .L2               //若 k 小于等于 0, 则转.L2
11   cmpw    %cx, %si          //R[si] - R[cx], 将 y 与 k 相减得到各标志
12   jg      .L1               //若 y 大于 k, 则转.L1
13  .L2:
14   movswl   %di, %eax        // R[edx]←R[di], 将 x*(y%k) 送 AX

```

10. 已知函数 sw()的 C 语言代码框架如下:

```

int sw(int x) {
    int v=0;
    switch (x) {
        /* switch 语句中的处理部分省略 */
    }
    return v;
}

```

对函数 sw()函数过程体中开始部分的 IA-32 汇编代码以及跳转表如下:

1	movl	8(%ebp), %eax	1	.L8:
2	addl	\$3, %eax	2	.long .L7
3	cmpl	\$7, %eax	3	.long .L2
4	ja	.L7	4	.long .L2
5	jmp	*.L8(, %eax, 4)	5	.long .L3
6	.L7:		6	.long .L4
7			7	.long .L5
8			8	.long .L7
			9	.long .L6

回答下列问题。

(1) 函数 sw()中的 switch 语句处理部分标号的取值情况如何?

(2) 标号的取值在什么情况下执行 default 分支? 哪些标号的取值会执行同一个 case 分支?

参考答案:

(1) 函数 sw()只有一个入口参数 x, 根据汇编代码的第 2~5 行指令知, 当 x+3>7 时转标号.L7 处执行, 否则, 按照跳转表中的地址转移执行, x 与跳转目标处标号的关系如下:

```

x+3=0: .L7
x+3=1: .L2
x+3=2: .L2
x+3=3: .L3
x+3=4: .L4
x+3=5: .L5

```

x+3=6: .L7

x+3=7: .L6

由此可知，switch (x) 中省略的处理部分结构如下：

```
case -2:
case -1:
    .....    // .L2 标号处指令序列对应的语句
    break;
case 0:
    .....    // .L3 标号处指令序列对应的语句
    break;
case 1:
    .....    // .L4 标号处指令序列对应的语句
    break;
case 2:
    .....    // .L5 标号处指令序列对应的语句
    break;
case 4:
    .....    // .L6 标号处指令序列对应的语句
    break;
default:
    .....    // .L7 标号处指令序列对应的语句
```

(2) 标号取值为-2、-1、0、1、2、4 以外的情况时，执行 default 分支。 -2 和-1 转同一个分支 (.L2) 执行。

11. 已知函数 test()的入口参数有 a、b、c 和 p，过程体代码如下：

```
*p = a;
return b*c;
```

函数 test()过程体对应的 IA-32 汇编代码如下：

```
1  movl    20(%ebp), %edx
2  movsbw  8(%ebp), %ax
3  movw    %ax, (%edx)
4  movzwl  12(%ebp), %eax
5  movzwl  16(%ebp), %ecx
6  mull    %ecx, %eax
```

完成下列任务。

(1) 写出函数 test()的原型，给出返回参数的类型以及入口参数 a、b、c 和 p 的类型和顺序。

(2) 写出对应的 x86-64 汇编代码。

参考答案：

(1) 根据第 2、3 行指令可知，参数 a 是 char 型，参数 p 是指向 short 型变量的指针；根据第 4、5 行指令可知，参数 b 和 c 都是 unsigned short 型，根据第 6 行指令可知，test 的返回参数类型为 unsigned int。因此，test 的原型为：unsigned int test(char a, unsigned short b, unsigned short c, short *p);

(2) 对应的 x86-64 汇编代码如下（入口参数 a, b, c 和 p 分别在 DI、SI、DX 和 RCX 中）：

```
1  movsbw  %dil, %ax
2  movw    %ax, (%rcx)
3  movzwl  %si, %eax
4  movzwl  %dx, %ecx
5  mull    %ecx, %eax
```

12. 已知函数 funct() 的 C 语言代码如下：

```
1  int funct(viod) {  
2      int  x, y;  
3      scanf("%x %x", &x, &y);  
4      return x-y;  
5  }
```

函数 funct() 对应的 IA-32 汇编代码如下：

```
1  funct:  
2  pushl    %ebp  
3  movl     %esp, %ebp  
4  subl     $40, %esp  
5  leal     -8(%ebp), %eax  
6  movl     %eax, 8(%esp)  
7  leal     -4(%ebp), %eax  
8  movl     %eax, 4(%esp)  
9  movl     $.LC0, (%esp)      //将指向字符串"%x %x"的指针入栈  
10 call     scanf              //假定 scanf 执行后 x=15, y=20  
11 movl     -4(%ebp), %eax  
12 subl     -8(%ebp), %eax  
13 leave  
14 ret
```

假设函数 funct() 开始执行时，R[esp]=0xbc00 0020，R[ebp]=0xbc00 0030，指向字符串"%x %x"的指针为 0x804c000。回答下列问题或完成下列任务。

- (1) 执行第 3、10 和 13 行的指令后，寄存器 EBP 中的内容分别是什么？
- (2) 执行第 3、10 和 13 行的指令后，寄存器 ESP 中的内容分别是什么？
- (3) 局部变量 x 和 y 所在存储单元的地址分别是什么？
- (4) 画出执行第 10 行指令后 funct 的栈帧，指出栈帧中的内容及其地址。

参考答案：

每次执行 pushl 指令后，R[esp]=R[esp]-4，因此，第 2 行指令执行后 R[esp]=0xbc00001c。

- (1) 执行第 3 行指令后，R[ebp]=R[esp]=0xbc00001c。到第 12 条指令执行结束都没有改变 EBP 的内容，因而执行第 10 行指令后，EBP 的内容还是为 0xbc00001c。执行第 13 行指令后，EBP 的内容恢复为进入函数 funct 时的值 0xbc000030。
- (2) 执行第 3 行指令后，R[esp]=0xbc00001c。执行第 4 行指令后 R[esp]=R[esp]-40=0xbc00001c-0x28=0xbbffff4。因而执行第 10 行指令后，未跳转到 scanf 函数执行时，ESP 中的内容为 0xbbffff4-4=0xbbffff0；在从 scanf 函数返回后 ESP 中的内容为 0xbbffff4。执行第 13 行指令后，ESP 的内容恢复为进入函数 funct 时的旧值，即 R[esp]=0xbc000020。
- (3) 第 5、6 两行指令将 scanf 的第三个参数 &y 入栈，入栈的内容为 R[ebp]-8=0xbc000014；第 7、8 两行指令将 scanf 的第二个参数 &x 入栈，入栈的内容为 R[ebp]-4=0xbc000018。故 x 和 y 所在的地址分别为 0xbc000018 和 0xbc000014。

(4) 执行第 10 行指令后，funct 栈帧的地址范围及其内容如下：

0xbc00001c	0xbc000030	← EBP 栈帧底部
0xbc000018	x=15	
0xbc000014	y=20	
0xbc000010		
0xbc00000c		
0xbc000008		
0xbc000004		
0xbc000000		
0xbbffffffc	0xbc000014	
0xbbffffff8	0xbc000018	
0xbbffffff4	0x804c000	
0xbbffffff0	从 scanf 返回的地址	← ESP

17. 假设函数 st_ele() 的 C 代码如下，其中，L、M 和 N 是用#define 声明的常数。

```

1  int a[L][M][N];
2
3  int st_ele(int i, int j, int k, int *dst) {
4      *dst = a[i][j][k];
5      return sizeof(a);
6  }
```

已知函数 st_ele() 的过程体对应的 IA-32 汇编代码如下：

```

1  movl    8(%ebp), %ecx
2  movl    12(%ebp), %edx
3  leal    (%edx,%edx, 8), %edx
4  movl    %ecx, %eax
5  sall    $6, %eax
6  subl    %ecx, %eax
7  addl    %eax, %edx
8  addl    16(%ebp), %edx
9  movl    a(, %edx, 4), %eax    //a 表示数组 a 的首地址
10 movl    20(%ebp), %edx
11 movl    %eax, (%edx)
12 movl    $4536, %eax
```

根据上述汇编代码，确定 L、M 和 N 的值，并写出函数 st_ele() 对应的 x86-64 汇编代码。

参考答案：

执行第 11 行指令后，a[i][j][k] 的地址为 $a + 4 * (63 * i + 9 * j + k)$ ，所以，可以推断出中间的 $M=9$ ， $N=63/9=7$ 。根据第 12 行指令，可知数组 a 的大小为 4536 字节，故 $L=4536/(4 * N * M)=18$ 。
函数 st_ele() 对应的 x86-64 汇编代码如下（参数 i, j, k 和 dst 分别在 EDI、ESI、EDX 和 RCX 中）：

```

1  movzq   %esi, %rsi
2  leaq     (%rsi, %rsi, 8), %rax
3  movzq   %edi, %rdi
4  movq     %rdi, %r10
5  salq     $6, %r10
```

```

6  subq    %rdi, %r10
7  addq    %rax, %r10
8  movzlb  %edx, %rdx
9  addl    %rdx, %r10
10 movl    a(, %r10, 4), %eax    //a 表示数组 a 的首地址
11 movl    %eax, (%rcx)
12 movl    $4536, %eax

```

20. 假设联合类型 utype 的定义如下：

```

typedef union {
    struct {
        int    x;
        short  y;
        short  z;
    } s1
    struct {
        short  a[2];
        int    b;
        char   *p;
    } s2
} utype;

```

若存在具有如下形式的一组函数：

```

void getvalue(utype *uptr, TYPE *dst) {
    *dst = EXPR;
}

```

该组函数用于计算不同表达式 EXPR 的值，返回值的数据类型根据表达式的类型确定。假设函数 getvalue() 的入口参数 uptr 和 dst 分别被装入寄存器 EAX 和 EDX 中，仿照例子填写下表，说明在不同的表达式下的 TYPE 类型以及表达式对应的 IA-32 汇编指令序列（要求尽量只用 EAX 和 EDX，不够用时再使用 ECX）。

表达式 EXPR	TYPE 类型	汇编指令序列
uptr->s1.x	int	movl (%eax), %eax movl %eax, (%edx)
uptr->s1.y		
&uptr->s1.z		
uptr->s2.a		
uptr->s2.a[uptr->s2.b]		
*uptr->s2.p		

参考答案：

表达式 EXPR	TYPE 类型	汇编指令序列
uptr->s1.x	int	movl (%eax), %eax movl %eax, (%edx)
uptr->s1.y	short	movw 4(%eax), %ax movw %ax, (%edx)
&uptr->s1.z	short *	leal 6(%eax), %eax movw %eax, (%edx)
uptr->s2.a	short *	movl %eax, (%edx)

uptr->s2.a[uptr->s2.b]	short	movl 4(%eax), %ecx movl (%eax, %ecx, 2), %eax movl %eax, (%edx)
*uptr->s2.p	char	movl 8(%eax), %eax movb (%eax), %al movb %al, (%edx)

21. 分别给出在 IA-32+Linux、x86-64+Linux 平台下，下列各个结构体类型中每个成员的偏移量、结构总大小以及结构起始位置的对齐要求。

- (1) struct S1 {short s; char c; int i; char d;};
- (2) struct S2 {int i; short s; char c; char d;};
- (3) struct S3 {char c; short s; int i; char d;};
- (4) struct S4 {short s[3]; char c;};
- (5) struct S5 {char c[3]; short *s; int i; char d; double e;};
- (6) struct S6 {struct S1 c[3]; struct S2 *s; char d;};

参考答案：

在 IA-32+Linux 平台下的情况如下。

- (1) S1: s c i d
 0 2 4 8 总共 12 字节，按 4 字节边界对齐
- (2) S2: i s c d
 0 4 6 7 总共 8 字节，按 4 字节边界对齐
- (3) S3: c s i d
 0 2 4 8 总共 12 字节，按 4 字节边界对齐
- (4) S4: s c
 0 6 总共 8 字节，按 2 字节边界对齐
- (5) S5: c s i d e
 0 4 8 12 16 总共 24 字节，按 4 字节边界对齐（Linux 下 double 型按 4 字节对齐）
- (6) S6: c s d
 0 36 40 总共 44 字节，按 4 字节边界对齐

在 x86-64+Linux 平台下的情况如下。

- (1) S1: s c i d
 0 2 4 8 总共 12 字节，按 4 字节边界对齐
- (2) S2: i s c d
 0 4 6 7 总共 8 字节，按 4 字节边界对齐
- (3) S3: c s i d
 0 2 4 8 总共 12 字节，按 4 字节边界对齐
- (4) S4: s c
 0 6 总共 8 字节，按 2 字节边界对齐
- (5) S5: c s i d e
 0 8 16 20 24 总共 32 字节，按 8 字节边界对齐（Linux 下 double 型按 8 字节对齐）
- (6) S6: c s d
 0 40 48 总共 56 字节，按 8 字节边界对齐

24. 假定函数 abc() 的入口参数有 a、b 和 c，每个参数都可能是带符号整数类型或无符号整数类型，而且它们的长度也可能不同。该函数具有如下过程体：

```
*b += c;
*a += *b;
```

在 x86-64 机器上编译后的汇编代码如下：

```
1  abc:
2      addl    (%rdx), %edi
3      movl    %edi, (%rdx)
4      movslq   %edi, %rdi
5      addq    %rdi, (%rsi)
6      ret
```

分析上述汇编代码，以确定三个入口参数的顺序和可能的数据类型，写出函数 `abc()` 可能的 4 种合理的函数原型。

参考答案：

x86-64 过程调用时参数传递是通过通用寄存器进行的，前三个参数所用寄存器顺序为 `RDI`、`RSI`、`RDX`。

`abc` 的 4 种合理的函数原型为：

- ① `viod abc(int c, long *a, int *b);`
- ② `viod abc(unsigned c, long *a, int *b);`
- ③ `viod abc(long c, long *a, int *b);`
- ④ `viod abc(unsigned long c, long *a, int *b);`

根据第 3、4 行指令可知，参数 `b` 肯定指向一个 32 位带符号整数类型；根据第 5 行指令可知，参数 `a` 指向 64 位带符号整数类型；而参数 `c` 可以是 32 位，也可以是 64 位，因为 `*b` 为 32 位，所以取 `RDI` 中的低 32 位 `R[edi]`（截断为 32 位），再和 `*b` 相加。同时，参数 `c` 可以是带符号整数类型，也可以是无符号整数类型，因为第 2 行加法指令 `addl` 的执行结果对于带符号整数和无符号整数都一样。

26. 假设你需要维护一个大型 C 语言程序，其部分代码如下：

```
1  typedef  struct {
2      unsigned    l_data;
3      line_struct  x[LEN];
4      unsigned    r_data;
5  } str_type;
6
7  void proc(int i, str_type *sptr) {
8      unsigned val = sptr->l_data + sptr->r_data;
9      line_struct *xptr = &sptr->x[i];
10     xptr->a[xptr->idx] = val;
11 }
```

编译时常量 `LEN` 以及结构类型 `line_struct` 的声明都在一个你无权访问的文件中，但是，你有代码的.o 版本（可重定位目标）文件，通过 `OBJDUMP` 反汇编该文件后，得到函数 `proc` 对应的反汇编结果如下，根据反汇编结果推断常量 `LEN` 的值以及结构类型 `line_struct` 的完整声明（假设其中只有成员 `a` 和 `idx`）。

1	00000000	<proc>:		
2	0:	55	push	%ebp
3	1:	89 e5	mov	%esp, %ebp
4	3:	53	push	%ebx
5	4:	8b 45 08	mov	0x8(%ebp), %eax
6	7:	8b 4d 0c	mov	0xc(%ebp), %ecx
7	a:	6b d8 1c	imul	\$0x1c, %eax, %ebx
8	d:	8d 14 c5 00 00 00 00	lea	0x0(, %eax, 8), %edx
9	14:	29 c2	sub	%eax, %edx
10	16:	03 54 19 04	add	0x4(%ecx, %ebx, 1), %edx
11	1a:	8b 81 c8 00 00 00	mov	0xc8(%ecx), %eax
12	20:	03 01	add	(%ecx), %eax
13	22:	89 44 91 08	mov	%eax, 0x8(%ecx, %edx, 4)
14	26:	5b	pop	%ebx
15	27:	5d	pop	%ebp
16	28:	c3	ret	

参考答案:

从第 5 行指令可知, i 在 EAX 中; 从第 6 行指令可知, sptr 在 ECX 中。由第 7 行指令可知, i*28 在 EBX 中。由第 8、9 和 10 行指令可猜出, x 的每个数组元素占 28B, 并且 xptr->idx 的地址为 sptr+i*28+4, 故在 line_struct 中的第一个分量为 idx, 因而后面的 24B 为 6 个数组元素 a[0]~a[5], 类型与 val 变量的类型相同, 即 unsigned int。

line_struct 结构类型的定义如下:

```
typedef struct {
    int      idx;
    unsigned a[6];
} line_struct;
```

由第 11、12 行指令可知, 数组 x 所占空间为 0xc8-4=200-4=196B, 所以 LEN=196/28=7。